2_ModulosYPaquetes

September 21, 2026

Creado por:

Isabel Maniega

1) Math
2) Random
3) Platform
4) Paquetes
5) Sys
6) PyPI

[1]:
```
from IPython import display
```

1 Módulos

Módulo es un archivo que contiene definiciones y sentencias de Python, que se pueden importar más tarde y utilizar cuando sea necesario.

Importar modulos

Para que un módulo sea utilizable, hay que importarlo (piensa en ello como sacar un libro del estante). La importación de un módulo se realiza mediante una instrucción llamada import. Nota: import es también una palabra clave reservada (con todas sus implicaciones).

2 Math

2.0.1 1) Ejemplo importar el modulo math de Python

[2]:
```
import math
```

Dentro del modulo math podemos importar la función seno para calcular el valor de pi entre dos:

[3]:
```
math.sin(math.pi/2)
```

[3]:
```
1.0
```

Lo realizamos llamando: **modulo + '.' + nombre de la entidad** ej. math.sin()

2.0.2 2) Otro ejemplo de importar modulo math en Python

[4]:
```
from math import sin, pi
```

- La palabra clave reservada **from**.
- El nombre del módulo a ser (selectivamente) importado.
- La palabra clave reservada **import**.
- El nombre o lista de nombres de la entidad o entidades las cuales estan siendo importadas al namespace.

[5]:
```
sin(pi/2)
```

[5]:
```
1.0
```

Se llaman directamente sin necesidad de declarar math de nuevo

2.0.3 3) Otro ejemplo de importar modulo math en Python

[6]:
```
from math import *
```

Esto quiere decir que importa todas las entidades que conforman el paquete **math**

2.1 Importando un módulo usando la palabra reservada *as*

[7]:
```
# pip install pandas
```

[8]:
```
import pandas as pd
```

La palabra **as** nos permite usar a lo largo del código la palabra asignada sin necesidad de usar el nombre completo. En este ejemplo **pandas** a partir de este momento pasará a llamarse **pd** a lo largo del código.

import modulo **as** alias

El "module" identifica el nombre del módulo original mientras que el "alias" es el nombre que se desea usar en lugar del original.

2.2 DIR

La función devuelve una lista ordenada alfabéticamente la cual contiene todos los nombres de las entidades disponibles en el módulo

[9]:
```
dir(math)
```

[9]:
```
['__doc__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 'acos',
 'acosh',
```

```
'asin',
'asinh',
'atan',
'atan2',
'atanh',
'cbrt',
'ceil',
'comb',
'copysign',
'cos',
'cosh',
'degrees',
'dist',
'e',
'erf',
'erfc',
'exp',
'exp2',
'expm1',
'fabs',
'factorial',
'floor',
'fmod',
'frexp',
'fsum',
'gamma',
'gcd',
'hypot',
'inf',
'isclose',
'isfinite',
'isinf',
'isnan',
'isqrt',
'lcm',
'ldexp',
'lgamma',
'log',
'log10',
'log1p',
'log2',
'modf',
'nan',
'nextafter',
'perm',
'pi',
'pow',
```

```
 'prod',
 'radians',
 'remainder',
 'sin',
 'sinh',
 'sqrt',
 'sumprod',
 'tan',
 'tanh',
 'tau',
 'trunc',
 'ulp']
```

math.pi → constant value.

math.e → Euler's number value.

math.sqrt(x) → the square root of x.

math.sin(x) → the sine of x.

math.cos(x) → the cosine of x.

math.tan(x) → the tangent of x.

math.asin(x) → the arcsine of x.

math.acos(x) → the arccosine of x.

math.atan(x) → the arctangent of x.

math.radians(x) → a function that converts x from degrees to radians.

math.degrees(x) → acting in the other direction (from radians to degrees).

math.sinh(x) → the hyperbolic sine.

math.cosh(x) → the hyperbolic cosine.

math.tanh(x) → the hyperbolic tangent.

math.asinh(x) → the hyperbolic arcsine.

math.acosh(x) → the hyperbolic arccosine.

math.atanh(x) → the hyperbolic arctangent.

math.exp(x) → finds the value of ex.

math.log(x) → the natural logarithm of x.

math.log(x, b) → the logarithm of x to base b.

math.log10(x) → the decimal logarithm of x (more precise than math.log(x, 10)).

log2(x) → the binary logarithm of x (more precise than math.log(x, 2)).

math.ceil(x) → the ceiling of x (the smallest integer greater than or equal to x).

math.floor(x) → the floor of x (the largest integer less than or equal to x).

math.trunc(x) → the value of x truncated to an integer (be careful – it's not an equivalent of either ceil or floor).

math.factorial(x) → returns x! (x has to be an integral and not a negative).

math.hypot(x, y) → returns the length of the hypotenuse of a right-angle triangle with the leg lengths equal to x and y (the same as math.sqrt(pow(x, 2) + pow(y, 2)) but more precise).

[10]:
```
dir(pd)
```

[10]:
```
['ArrowDtype',
 'BooleanDtype',
 'Categorical',
 'CategoricalDtype',
 'CategoricalIndex',
 'DataFrame',
 'DateOffset',
 'DatetimeIndex',
 'DatetimeTZDtype',
 'ExcelFile',
 'ExcelWriter',
 'Flags',
 'Float32Dtype',
 'Float64Dtype',
 'Grouper',
 'HDFStore',
 'Index',
 'IndexSlice',
 'Int16Dtype',
 'Int32Dtype',
 'Int64Dtype',
 'Int8Dtype',
 'Interval',
 'IntervalDtype',
 'IntervalIndex',
 'MultiIndex',
 'NA',
 'NaT',
 'NamedAgg',
 'Period',
 'PeriodDtype',
 'PeriodIndex',
 'RangeIndex',
 'Series',
 'SparseDtype',
 'StringDtype',
 'Timedelta',
```

```
'TimedeltaIndex',
'Timestamp',
'UInt16Dtype',
'UInt32Dtype',
'UInt64Dtype',
'UInt8Dtype',
'__all__',
'__builtins__',
'__cached__',
'__doc__',
'__docformat__',
'__file__',
'__git_version__',
'__loader__',
'__name__',
'__package__',
'__path__',
'__spec__',
'__version__',
'_built_with_meson',
'_config',
'_is_numpy_dev',
'_libs',
'_pandas_datetime_CAPI',
'_pandas_parser_CAPI',
'_testing',
'_typing',
'_version_meson',
'annotations',
'api',
'array',
'arrays',
'bdate_range',
'col',
'compat',
'concat',
'core',
'crosstab',
'cut',
'date_range',
'describe_option',
'errors',
'eval',
'factorize',
'from_dummies',
'get_dummies',
'get_option',
```

```
'infer_freq',
'interval_range',
'io',
'isna',
'isnull',
'json_normalize',
'lreshape',
'melt',
'merge',
'merge_asof',
'merge_ordered',
'notna',
'notnull',
'offsets',
'option_context',
'options',
'pandas',
'period_range',
'pivot',
'pivot_table',
'plotting',
'qcut',
'read_clipboard',
'read_csv',
'read_excel',
'read_feather',
'read_fwf',
'read_hdf',
'read_html',
'read_iceberg',
'read_json',
'read_orc',
'read_parquet',
'read_pickle',
'read_sas',
'read_spss',
'read_sql',
'read_sql_query',
'read_sql_table',
'read_stata',
'read_table',
'read_xml',
'reset_option',
'set_eng_float_format',
'set_option',
'show_versions',
'test',
```

```
 'testing',
 'timedelta_range',
 'to_datetime',
 'to_numeric',
 'to_pickle',
 'to_timedelta',
 'tseries',
 'unique',
 'util',
 'wide_to_long']
```

¿Has notado los nombres extraños que comienzan con ___ al inicio de la lista? Se hablará más sobre ellos cuando hablemos sobre los problemas relacionados con la escritura de módulos propios.

[11]:
```
# the floor of x (the largest integer less than or equal to x).

math.floor(3.999999)
```

[11]: 3

[12]:
```
#  the ceiling of x (the smallest integer greater than or equal to x).

math.ceil(3.000000001)
```

[12]: 4

[13]:
```
# Returns the value of x raised to power y.
# This method converts both arguments into a float.

math.pow(2, 3)
```

[13]: 8.0

[14]:
```
# If 'x' is negative and 'y' is not an integer, it returns a ValueError.

math.pow(-2, 1.2)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[14], line 3
      1 # If 'x' is negative and 'y' is not an integer, it returns a ValueError
      2
----> 3 math.pow(-2, 1.2)

ValueError: math domain error
```

[15]:
```
# the square root of x.

math.sqrt(8)
```

[15]:
```
2.8284271247461903
```

[16]:
```
# the square root of x.

math.sqrt(-8)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[16], line 3
      1 # the square root of x.
      2
----> 3 math.sqrt(-8)

ValueError: math domain error
```

[17]:
```
#   constant value.

math.pi
```

[17]:
```
3.141592653589793
```

[18]:
```
value = math.pi < math.floor(3.14)
value
```

[18]:
```
False
```

[19]:
```
value = math.pi < math.ceil(3.14)
# value = 3.14 < 4
# True => 1
int(value)
```

[19]:
```
1
```

[20]:
```
# returns x! (x has to be an integral and not a negative).

math.factorial(5)
```

[20]:
```
120
```

[21]:
```
# returns x! (x has to be an integral and not a negative).

math.factorial(-5)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[21], line 3
      1 # returns x! (x has to be an integral and not a negative).
      2
----> 3 math.factorial(-5)

ValueError: factorial() not defined for negative values
```

[22]:
```
math.floor(3.00), math.ceil(3.00)
```

[22]:
```
(3, 3)
```

[23]:
```
print(math.floor(-15.9999999))
print(math.floor(-15.0000001))
```

```
-16
-16
```

[24]:
```
#set perpendicular and base
parendicular = 3
base = 4

#print the hypotenuse of a right-angled triangle
print(math.hypot(parendicular, base))
```

```
5.0
```

[25]:
```
# Return the truncated integer parts of different numbers

print(math.trunc(18.77))
print(math.trunc(25.343))
print(math.trunc(-991.5))
```

```
18
25
-991
```

3 Random

Nos permite obtener número **pseudoaleatorios**, esto quiere decir que los algoritmos, que generan los números, no son aleatorios, son deterministas y predecibles.

Los números random, necesitan determinar una **semilla**, calcula un número "aleatorio" basado en él (el método depende de un algoritmo elegido). El valor de la semilla inicial, establecido durante el inicio del programa, determina el orden en que aparecerán los valores generados.

3.0.1 Ejemplo 1:

Generamos números pseudoaleatorios de 0.0 a 1.0

[26]:
```
from random import random

for i in range(5):
    print(random())
```

```
0.26776152826844435
0.8832529778358689
0.052609966859991975
0.2586008195135492
0.2447629830041015
```

La semilla en este caso no está definida, por lo tanto es dificil saber porque valor empieza y presentará una aleatoriedad.

3.0.2 Ejemplo 2: seed

Añadimos una semilla de 0

[27]:
```
from random import random, seed

seed(0)

for i in range(5):
    print(random())
```

```
0.8444218515250481
0.7579544029403025
0.420571580830845
0.25891675029296335
0.5112747213686085
```

Debido al hecho de que la semilla siempre se establece con el mismo valor, la secuencia de valores generados siempre se ve igual.

3.0.3 Ejemplo 3: randrange, randint

randrange y ranint: genera números enteros aleatorios en un rango determinado:.

- randrange(inicio, fin, incremento). Ejemplo de 0 a 2 (No incluido)
- randint(izquierda, derecha). Ejemplo de 0 a 2 (incluido)

[28]:
```
from random import randrange, randint

print(randrange(100), end=' ')
print(randrange(0, 100), end=' ')
print(randrange(0, 100, 1), end=' ')
print(randint(0, 100))
```

```
51 38 61 45
```

Nota: Observa que los datos generados no son único

[29]:
```
from random import randint

for i in range(10):
    print(randint(1, 10), end=', ')
```

```
10, 4, 9, 3, 5, 3, 2, 10, 5, 9,
```

3.0.4 Ejemplo 4: Choice, sample

- choice(secuencia)
- sample(secuencia, elementos_a_elegir=1)

choice elige un elemento "aleatorio" de la secuencia de entrada y lo devuelve.

sample crea una lista (una muestra) que consta del elemento elementos_a_elegir (que por defecto es 1) "sorteado" de la secuencia de entrada.

[30]:
```
from random import choice, sample

my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

print(choice(my_list))
print(sample(my_list, 5))
print(sample(my_list, 10))
```

```
10
[3, 5, 2, 6, 1]
[6, 8, 2, 3, 4, 7, 9, 5, 10, 1]
```

3.0.5 Ejemplo 5: Shuffle

[31]:
```
from random import shuffle

my_list2 = [4, 5, 8, 14, 95, 74, 41]

# Reorganiza los elementos aleatoriamente:
shuffle(my_list2)

my_list2
```

[31]:
```
[5, 14, 74, 4, 8, 95, 41]
```

4 Estructura de ejecución de código

- Tu código quiere crear un archivo, por lo que invoca una de las funciones de Python.

- Python acepta la orden, la reorganiza para cumplir con los requisitos del sistema operativo local, es como poner el sello "aprobado" en una solicitud y lo envía (esto puede recordarte una cadena de mando).
- El SO comprueba si la solicitud es razonable y válida (por ejemplo, si el nombre del archivo se ajusta a algunas reglas de sintaxis) e intenta crear el archivo. Tal operación, aparentemente es muy simple, no es atómica: consiste de muchos pasos menores tomados por:
- El hardware, el cual es responsable de activar los dispositivos de almacenamiento (disco duro, dispositivos de estado sólido, etc.) para satisfacer las necesidades del sistema operativo.

4.1 Módulo Platform

El módulo platform permite acceder a los datos de la plataforma subyacente, es decir, hardware, sistema operativo e información sobre la versión del intérprete.

Simplemente devuelve una cadena que describe el entorno; por lo tanto, su salida está más dirigida a los humanos que al procesamiento automatizado (lo verás pronto).

[32]:
```
from platform import platform

print(platform())
print(platform(1))
print(platform(0, 1))
```

```
Linux-6.8.0-139-generic-x86_64-with-glibc2.39
Linux-6.8.0-139-generic-x86_64-with-glibc2.39
Linux-6.8.0-139-generic-x86_64-with-glibc2.39
```

- aliased → cuando se establece a True (o cualquier valor distinto a cero) puede hacer que la función presente los nombres de capa subyacentes alternativos en lugar de los comunes.
- terse → cuando se establece a True (o cualquier valor distinto a cero) puede convencer a la función de presentar una forma más breve del resultado (si lo fuera posible).

4.1.1 1) machine()

Para conocer el nombre genérico del procesador que ejecuta el sistema operativo junto con Python y el código, una función llamada machine() te lo dirá.

[33]:
```
from platform import machine

print(machine())
```

```
x86_64
```

4.1.2 2) processor()

La función processor() devuelve una cadena con el nombre real del procesador (si lo fuese posible).

[34]:
```
from platform import processor

print(processor())
```

```
x86_64
```

4.1.3 3) System

Una función llamada system() devuelve el nombre genérico del sistema operativo en una cadena.

[35]:
```
from platform import system

print(system())
```

```
Linux
```

4.1.4 4) version()

La versión del sistema operativo se proporciona como una cadena por la función version().

[36]:
```
from platform import version

print(version())
```

```
#139-Ubuntu SMP PREEMPT_DYNAMIC Sat Aug  1 03:52:05 UTC 2026
```

4.1.5 5) python_implementation y python_version_tuple

Las funciones python_implementation y python_version_tuple

Si necesitas saber que versión de Python está ejecutando tu código, puedes verificarlo utilizando una serie de funciones dedicadas, aquí hay dos de ellas:

- **python_implementation()** → devuelve una cadena que denota la implementación de Python (espera CPython aquí, a menos que decidas utilizar cualquier rama de Python no canónica).
- **python_version_tuple()** → devuelve una tupla de tres elementos la cual contiene, ej 3.8.18:
  - La parte mayor de la versión de Python.
  - La parte menor.
  - El número del nivel de parche.

[37]:
```
from platform import python_implementation, python_version_tuple

print(python_implementation())

for atr in python_version_tuple():
    print(atr)
```

```
CPython
3
12
3
```

5 Paquetes

- Un módulo es un contenedor lleno de funciones - puedes empaquetar tantas funciones como desees en un módulo y distribuirlo por todo el mundo.
- Por supuesto, no es una buena idea mezclar funciones con diferentes áreas de aplicación dentro de un módulo (al igual que en una biblioteca: nadie espera que los trabajos científicos se incluyan entre los cómics), así que se deben agrupar las funciones cuidadosamente y asignar un nombre claro e intuitivo al módulo que las contiene (por ejemplo, no le des el nombre videojuegos a un módulo que contiene funciones destinadas a particionar y formatear discos duros).
- Crear muchos módulos puede causar desorden: tarde que temprano querrás agrupar tus módulos de la misma manera que previamente has agrupado funciones: ¿Existe un contenedor más general que un módulo?
- Sí lo hay, es un paquete: en el mundo de los módulos, un paquete juega un papel similar al de una carpeta o directorio en el mundo de los archivos.

```
[any directory]


        package             ← directory

            gearbox.py      ← file: contains turn_on() function

            __init__.py     ← empty file

            subpackage      ← directory

                engine.py   ← file: contains start() function
```

Podemos importar script `gearbox.py` y su función `turn_on()` de la siguiente forma:

```
import package.gearbox

package.gearbox.turn_on()
```

Otra forma de importar la función es:

```
from package.gearbox import turn_on

turn_on()
```

Otra forma más de importarlo:

```
from package.gearbox import *

start()
```

5.0.1 1) Crear un modulo

[38]:
```
# 1) Crea un script con nombre module.py
# 2) Crea un script con nombre main.py
# 3) Dentro del script main, llama al script module:
    # import module
# 4) Ejecuta el script main.py y no deberias de ver como salida nada,
# si no ha realizado ningún error, ha realizado con éxito la importación del␣
 ↪modulo (module.py)
```

Al ejecutar el script main.py verás que ha aparecido una nueva subcarpeta, ¿puedes verla? Su nombre es **pycache**. Echa un vistazo adentro. ¿Qué es lo que ves?

Hay un archivo llamado (más o menos) module.cpython-xy.pyc donde x y y son dígitos derivados de tu versión de Python (por ejemplo, serán 3 y 8 si utilizas Python 3.8).

El nombre del archivo es el mismo que el de tu módulo. La parte posterior al primer punto dice qué implementación de Python ha creado el archivo (CPython) y su número de versión. La ultima parte (pyc) viene de las palabras Python y compilado.

Puedes mirar dentro del archivo: el contenido es completamente ilegible para los humanos. Tiene que ser así, ya que el archivo está destinado solo para uso el uso de Python.

Cuando Python importa un módulo por primera vez, traduce el contenido a una forma algo compilada.

El archivo no contiene código en lenguaje máquina: es código semi-compilado interno de Python, listo para ser ejecutado por el intérprete de Python. Como tal archivo no requiere tantas comprobaciones como las de un archivo fuente, la ejecución comienza más rápido y también se ejecuta más rápido.

Gracias a eso, cada importación posterior será más rápida que interpretar el código fuente desde cero.

Python puede verificar si el archivo fuente del módulo ha sido modificado (en este caso, el archivo pyc será reconstruido) o no (cuando el archivo pyc pueda ser ejecutado al instante). Este proceso es completamente automático y transparente, no tiene que ser tomando en cuenta.

5.0.2 2) mostrar la información de module.py

[39]:
```
# Podemos repetir los pasos anteriores pero en este punto añadimos al script␣
 ↪module.py un print,
# por los tanto los pasos serían:

# 1) Crea un script con nombre mudule.py
# 2) Dentro del script module.py realiza un print:
    # print("Me gusta ser un módulo.")
# 3) Crea un script con nombre main.py
# 4) Dentro del script main, llama al script module:
    # import module
# 5) Ejecuta el script main.py en este punto verás que la salida muestra:
```

```
    # Me gusta ser un módulo.
# Visualizamos el contenido del módulo module.py
```

5.0.3 3) '__ __ *name* __ __'

[40]:
```
# Podemos repetir los pasos anteriores pero en este punto añadimos al script␣
 ↪module.py un print que muestre '__name__',
# por los tanto los pasos serían:

# 1) Crea un script con nombre module.py
# 2) Dentro del script module.py realiza un print:
    # print("Me gusta ser un módulo.")
    # print(__name__)
# 3) Crea un script con nombre main.py
# 4) Dentro del script main, llama al script module:
    # import module
# 5) Ejecuta el script main.py en este punto verás que la salida muestra:
    # Me gusta ser un módulo.
    # __main__ ó module (si es un modulo como es este ejemplo)

# Podemos decir que:

    # Cuando se ejecuta un archivo directamente, su variable __name__ se␣
 ↪establece a __main__.
    # Cuando un archivo se importa como un módulo, su variable __name__ se␣
 ↪establece al nombre del archivo (excluyendo a .py).
```

5.0.4 4) main

[41]:
```
# Podemos repetir los pasos anteriores pero en este punto añadimos al script␣
 ↪module.py un print que muestre '__name__',
# por los tanto los pasos serían:

# 1) Crea un script con nombre mudule.py
# 2) Dentro del script module.py realiza un print:
    # print("Me gusta ser un módulo.")
    # if __name__ == "__main__":
        # print("Yo prefiero ser un módulo")
    # else:
        # print("Me gusta ser un módulo")
# 3) Crea un script con nombre main.py
# 4) Dentro del script main, llama al script module:
    # import module
# 5) Ejecuta el script main.py en este punto verás que la salida muestra:
    # Me gusta ser un módulo.
    # __main__
```

```
# Podemos decir que:

    # Cuando se ejecuta un archivo directamente, su variable __name__ se␣
 ↪establece a __main__.
    # Cuando un archivo se importa como un módulo, su variable __name__ se␣
 ↪establece al nombre del archivo (excluyendo a .py).
```

5.0.5 5) Contador de llamada de funciones

[42]:
```
# Podemos repetir los pasos anteriores pero en este punto añadimos al script␣
 ↪module.py un print que muestre '__name__',
# por los tanto los pasos serían:

# 1) Crea un script con nombre mudule.py
# 2) Dentro del script module.py realiza un print:
    # count = 0
    # if __name__ == "__main__":
        # print("Yo prefiero ser un módulo")
    # else:
        #print("Me gusta ser un módulo")
# 3) Crea un script con nombre main.py
# 4) Dentro del script main, llama al script module:
    # import module
    # print(module.counter)
# 5) Ejecuta el script main.py en este punto verás que la salida muestra:
    # Yo prefiero ser un módulo.
    # 0

# Como puedes ver, el archivo principal intenta acceder a la variable de␣
 ↪contador del módulo.
# ¿Es esto legal? Sí lo es. ¿Es utilizable? Claro. ¿Es seguro?

# Eso depende: si confías en los usuarios de tu módulo, no hay problema;
# sin embargo, es posible que no desees que el resto del mundo vea tu variable␣
 ↪personal o privada.

# A diferencia de muchos otros lenguajes de programación,
# Python no tiene medios para permitirte ocultar tales variables a los ojos de␣
 ↪los usuarios del módulo.

# Solo puedes informar a tus usuarios que esta es tu variable, que pueden␣
 ↪leerla,
# pero que no deben modificarla bajo ninguna circunstancia.

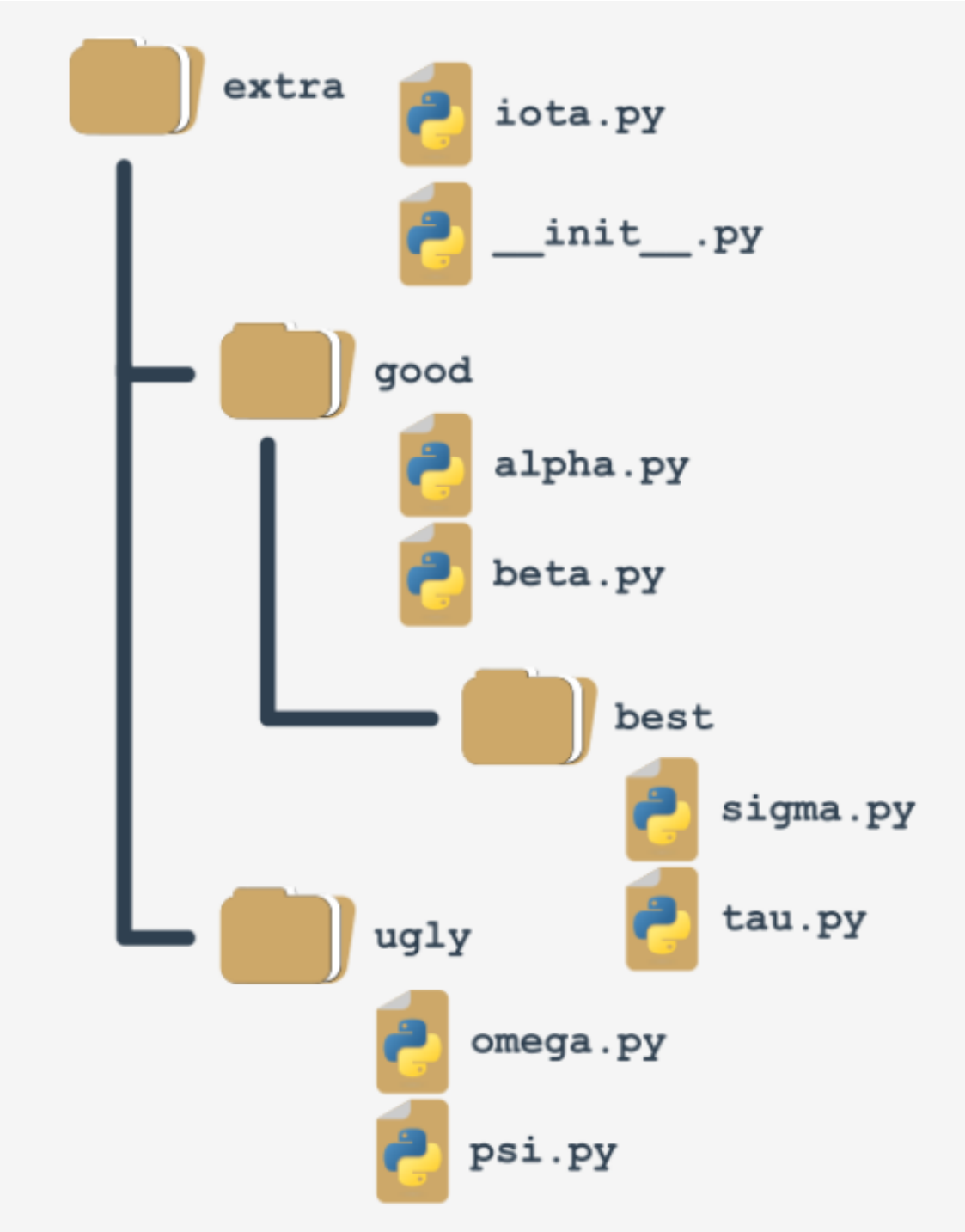
# Esto se hace anteponiendo al nombre de la variable _ (un guión bajo) o __␣
 ↪(dos guiones bajos),
```

```
# pero recuerda, es solo un acuerdo. Los usuarios de tu módulo pueden␣
  ↪obedecerlo o no.
```

5.0.6 6) Árbol carpetas

[44]:
```
display.Image("./images/arbol_carpetas.png")
```

[44]:



Nota: no solo la carpeta raiz puede contener el archivo **init.py**, también puedes ponerlo dentro de cualquiera de sus subcarpetas (subpaquetes). Puede ser útil si algunos de los subpaquetes requieren tratamiento individual o un tipo especial de inicialización.

Para acceder a la función funT() ubicado en el archivo tau pondremos:

```
extra.good.best.tau.funT()
```

Y para acceder a la función funP() del archivo psi:

```
extra.ugly.psi.funP()
```

Entonces para importarlo como sería:

```
import extra.good.best.tau as t
import extra.ugly.psi as p
```

otra forma:

```
from extra.good.best.tau import funT
from extra.ugly.psi import funP

print(t.funT())
print(p.funP())
print(funT())
print(funP())
```

Los nombres shabang, shebang, hasbang, poundbang y hashpling describen el dígrafo escrito como **#!**, se utiliza para instruir a los sistemas operativos similares a Unix sobre cómo se debe iniciar el archivo fuente de Python. Esta convención no tiene efecto en MS Windows.

6 Modulo sys

[45]:
```
import sys

for p in sys.path:
    print(p)
```

```
/usr/lib/python312.zip
/usr/lib/python3.12
/usr/lib/python3.12/lib-dynload

/home/isabelmaniega/Documentos/Programacion Python 20h/venv/lib/python3.12/site-
packages
```

Ten en cuenta también que: hay un archivo zip listado como uno de los elementos de la ruta, esto no es un error. Python puede tratar los archivos zip como carpetas ordinarias, esto puede ahorrar mucho almacenamiento.

Deseas evitar que el usuario de tu módulo ejecute tu código como un script ordinario. ¿Cómo lograrías tal efecto?

[46]:
```
import sys

if __name__ == "__main__":
    print("¡No hagas eso!")
    sys.exit()
```

```
¡No hagas eso!
```

```
An exception has occurred, use %tb to see the full traceback.

SystemExit
```

```
/home/isabelmaniega/Documentos/Programacion Python 20h/venv/lib/python3.12/site-
packages/IPython/core/interactiveshell.py:3831: UserWarning: To exit: use
'exit', 'quit', or Ctrl-D.
  warn("To exit: use 'exit', 'quit', or Ctrl-D.", stacklevel=1)
```

[47]:
```
# Ejecutamos el script index.py: python index.py Hello 2

# from sys import argv

# print(argv[0])

# Output: index.py
```

7 Pypi

El repositorio de Python es **PyPI** (es la abreviatura de Python Package Index) y lo mantiene un grupo de trabajo llamado Packaging Working Group, una parte de la Python Software Foundation, cuya tarea principal es apoyar a los desarrolladores de Python en la diseminación de código eficiente.

https://wiki.python.org/psf/PackagingWG

https://pypi.org/

pip es package installer for Python (https://es.wikipedia.org/wiki/Pip_(administrador_de_paquetes))

Para descargar los paquetes del repositorio de Pypi necesitas usar *pip*.

Para verificar su instalación usamos:

- pip3 –version –> Si tenemos python 2 instalado en el sistema
- pip –version

Para pedir ayuda a pip se realiza con:

- pip help

[48]:
```
pip --version
```

```
pip 24.0 from /home/isabelmaniega/Documentos/Programacion Python
20h/venv/lib/python3.12/site-packages/pip (python 3.12)
Note: you may need to restart the kernel to use updated packages.
```

[49]:
```
pip help
```

```
Usage:
  /home/isabelmaniega/Documentos/Programacion Python 20h/venv/bin/python -m pip
<command> [options]

Commands:
  install                     Install packages.
  download                    Download packages.
  uninstall                   Uninstall packages.
  freeze                      Output installed packages in requirements format.
  inspect                     Inspect the python environment.
  list                        List installed packages.
  show                        Show information about installed packages.
  check                       Verify installed packages have compatible
dependencies.
  config                      Manage local and global configuration.
  search                      Search PyPI for packages.
  cache                       Inspect and manage pip's wheel cache.
  index                       Inspect information available from package
indexes.
  wheel                       Build wheels from your requirements.
  hash                        Compute hashes of package archives.
  completion                  A helper command used for command completion.
  debug                       Show information useful for debugging.
  help                        Show help for commands.

General Options:
  -h, --help                  Show help.
  --debug                     Let unhandled exceptions propagate outside the
                              main subroutine, instead of logging them to
                              stderr.
  --isolated                  Run pip in an isolated mode, ignoring
                              environment variables and user configuration.
  --require-virtualenv        Allow pip to only run in a virtual environment;
                              exit with an error otherwise.
  --python <python>           Run pip with the specified Python interpreter.
  -v, --verbose               Give more output. Option is additive, and can be
                              used up to 3 times.
  -V, --version               Show version and exit.
  -q, --quiet                 Give less output. Option is additive, and can be
```

```
                              used up to 3 times (corresponding to WARNING,
                              ERROR, and CRITICAL logging levels).
  --log <path>                Path to a verbose appending log.
  --no-input                  Disable prompting for input.
  --keyring-provider <keyring_provider>
                              Enable the credential lookup via the keyring
                              library if user input is allowed. Specify which
                              mechanism to use [disabled, import, subprocess].
                              (default: disabled)
  --proxy <proxy>             Specify a proxy in the form
                              scheme://[user:passwd@]proxy.server:port.
  --retries <retries>         Maximum number of retries each connection should
                              attempt (default 5 times).
  --timeout <sec>             Set the socket timeout (default 15 seconds).
  --exists-action <action>    Default action when a path already exists:
                              (s)witch, (i)gnore, (w)ipe, (b)ackup, (a)bort.
  --trusted-host <hostname>   Mark this host or host:port pair as trusted,
                              even though it does not have valid or any HTTPS.
  --cert <path>               Path to PEM-encoded CA certificate bundle. If
                              provided, overrides the default. See 'SSL
                              Certificate Verification' in pip documentation
                              for more information.
  --client-cert <path>        Path to SSL client certificate, a single file
                              containing the private key and the certificate
                              in PEM format.
  --cache-dir <dir>           Store the cache data in <dir>.
  --no-cache-dir              Disable the cache.
  --disable-pip-version-check
                              Don't periodically check PyPI to determine
                              whether a new version of pip is available for
                              download. Implied with --no-index.
  --no-color                  Suppress colored output.
  --no-python-version-warning
                              Silence deprecation warnings for upcoming
                              unsupported Pythons.
  --use-feature <feature>     Enable new functionality, that may be backward
                              incompatible.
  --use-deprecated <feature>  Enable deprecated functionality, that will be
                              removed in the future.
Note: you may need to restart the kernel to use updated packages.
```

Para saber los paquetes instalados usamos:

- pip list

[50]:
```
pip list
```

```
Package                   Version
------------------------- -----------
```

```
anyio                       4.15.1
argon2-cffi                 25.1.0
argon2-cffi-bindings        26.1.0
arrow                       1.4.0
asttokens                   3.0.2
async-lru                   2.3.0
attrs                       26.1.0
babel                       2.18.0
beautifulsoup4              4.15.0
bleach                      6.4.0
certifi                     2026.7.22
cffi                        2.1.1
charset-normalizer          3.5.1
comm                        0.2.3
contourpy                   1.4.0
cycler                      0.12.1
debugpy                     1.8.22
defusedxml                  0.7.1
executing                   2.2.1
fastjsonschema              2.22.2
fonttools                   4.65.0
fqdn                        1.5.1
h11                         0.16.0
httpcore                    1.0.9
httpx                       0.28.1
idna                        3.19
ipykernel                   7.3.0
ipython                     9.17.1
ipython_pygments_lexers     1.1.1
isoduration                 20.11.0
jedi                        0.20.0
Jinja2                      3.1.6
json5                       0.15.0
jsonpointer                 3.1.1
jsonschema                  4.26.0
jsonschema-specifications   2025.9.1
jupyter_builder             1.2.3
jupyter_client              8.10.0
jupyter_core                5.9.1
jupyter-events              0.12.1
jupyter-lsp                 2.3.1
jupyter_server              2.21.1
jupyter_server_terminals    0.5.4
jupyterlab                  4.6.3
jupyterlab_pygments         0.3.0
jupyterlab_server           2.28.1
kiwisolver                  1.5.1
lark                        1.3.1
```

MarkupSafe	3.0.3
matplotlib	3.11.2
matplotlib-inline	0.2.2
mistune	3.3.4
nbclient	0.11.0
nbconvert	7.17.1
nbformat	5.11.1
nest-asyncio2	1.7.2
notebook	7.6.2
notebook_shim	0.2.4
numpy	2.5.3
packaging	26.3
pandas	3.0.6
pandocfilters	1.5.1
parso	0.8.7
pexpect	4.9.0
pillow	12.3.0
pip	24.0
platformdirs	4.11.9
prometheus_client	0.26.0
prompt_toolkit	3.0.53
psutil	7.2.2
ptyprocess	0.7.0
pure_eval	0.2.4
pycparser	3.0
Pygments	2.21.0
pyparsing	3.3.3
python-dateutil	2.9.0.post0
python-json-logger	4.2.0
PyYAML	6.0.3
pyzmq	27.2.0
referencing	0.37.0
requests	2.34.2
rfc3339-validator	0.1.4
rfc3986-validator	0.1.1
rfc3987-syntax	1.1.0
rpds-py	2026.6.3
Send2Trash	2.1.0
six	1.17.0
soupsieve	2.9.2
stack-data	0.6.3
terminado	0.18.1
tinycss2	1.5.1
tornado	6.5.10
traitlets	5.16.1
typing_extensions	4.16.0
tzdata	2026.4
uri-template	1.3.0

```
urllib3                   2.8.0
wcwidth                   0.8.3
webcolors                 25.10.0
webencodings              0.6.1
websocket-client          1.9.2
Note: you may need to restart the kernel to use updated packages.
```

Para mostrar más información sobre un paquete:

- pip show nombre del paquete

[51]:
```
pip show pip
```

```
Name: pip
Version: 24.0
Summary: The PyPA recommended tool for installing Python packages.
Home-page:
Author:
Author-email: The pip developers <distutils-sig@python.org>
License: MIT
Location: /home/isabelmaniega/Documentos/Programacion Python
20h/venv/lib/python3.12/site-packages
Requires:
Required-by:
Note: you may need to restart the kernel to use updated packages.
```

Para buscar un paquete determinado:

- pip search anystring

[52]:
```
 #pip search pip
# Da un error en jupyter
```

pip emplea una opción dedicada llamada –user (observa el guión doble). La presencia de esta opción indica a pip que actúe localmente en nombre de tu usuario sin privilegios de administrador.

Como administrador la instalación es: pip install pygame Como usuario sin derechos de administrador es: pip install –user pygame

El comando **pip install** tiene dos habilidades adicionales importantes:

Es capaz de actualizar un paquete instalado localmente; por ejemplo, si deseas asegurarte de que estás utilizando la última versión de un paquete en particular, puedes ejecutar el siguiente comando:

pip install -U nombre_del_paquete

Es capaz de instalar una versión seleccionada por el usuario de un paquete (pip instala por defecto la versión más nueva disponible); para lograr este objetivo debes utilizar la siguiente sintaxis:

pip install nombre_del_paquete==versión_del_paquete

Si alguno de los paquetes instalados actualmente ya no es necesario y deseas deshacerte de el, pip también será útil. Su comando uninstall ejecutará todos los pasos necesarios.

pip uninstall nombre_del_paquete

Creado por:

Isabel Maniega